Containerized Monitoring & ML Anomaly System

Complete Step-by-Step VS Code Runbook & Architecture Verification Guide

1. System Architecture & Overview

This project implements an end-to-end predictive error detection layer on top of a containerized application monitoring stack. Locust generates synthetic user traffic (Normal, Ramp, Spike, Heavy Load), Prometheus collects system metrics, an Isolation Forest ML service evaluates metric vectors to produce real-time anomaly scores, Alertmanager triggers predictive alerts, and a React Developer Dashboard displays live health scores.

Locust Traffic Generator	→	**Flask Web Application**	→	**Prometheus Metrics**
Prometheus Metrics	→	**ML Isolation Forest**	→	**Predictive Anomaly Score**
Anomaly Score (0.0-1.0)	→	**Alertmanager Warnings**	→	**React / Grafana Dashboard**

2. Prerequisites & Tools Required

Tool	Required Version	Purpose
VS Code	Latest	Development & Terminal Execution
Python	3.10+	Flask App & Isolation Forest ML Service
Node.js / npm	18+	React Developer Dashboard (Vite)
Docker Desktop	Latest	Containerized Multi-Service Deployment
Git	Latest	Version Control & GitHub Sync

3. Step-by-Step Execution Guide in VS Code

Step 1: Open Project in VS Code

1. Open VS Code.

2. Select File > Open Folder... and choose: c:\Users\aksha\Downloads\containerized-monitoring-system-main

3. Open integrated terminal using Ctrl + ~.

Step 2: Train Model & Launch ML Microservice (Port 5001)

```
1. In VS Code terminal, navigate to ml_service: cd ml_service
```

2. Install dependencies: pip install -r requirements.txt

```
3. Train Isolation Forest model: python train.py
```

```
4. Start ML prediction service: python app.py
→ ML Service listens at http://localhost:5001
```

Step 3: Start Application Backend (Port 5000)

1. Open a new terminal tab in VS Code (Ctrl + Shift + ~).

```
2. Run Flask application: python app/src/app.py
→ Backend API listens at http://localhost:5000
```

Step 4: Launch React Developer Dashboard (Port 5173)

1. Open a new terminal tab in VS Code.

```
2. Navigate to frontend folder: cd frontend
3. Install node packages: npm install
4. Start Vite dev server: npm run dev
→ Open browser at http://localhost:5173
```

Step 5: Run Full Docker Monitoring Stack

1. Ensure Docker Desktop is running.

```
2. In root terminal, execute: docker-compose up --build
→ Builds and starts Flask App, ML Service, Prometheus, Alertmanager, Grafana.
```

Step 6: Run Locust Load Testing

```
1. In a new terminal tab, navigate to load-testing/locust: cd load-testing/locust
```

2. Launch Locust: locust -f locustfile.py --host=http://localhost:5000

3. Open http://localhost:8089, set 50 Users, spawn rate 5, and click Start Swarm.

4. Port & Service Directory Cheat Sheet

Service	Port	URL	Description
Flask App	5000	http://localhost:5000	Backend API & Prometheus Metrics
ML Service	5001	http://localhost:5001	Isolation Forest ML Prediction Microservice
React Dashboard	5173	http://localhost:5173	ML Intelligence Dashboard & Simulator
Prometheus	9090	http://localhost:9090	Time-series Metrics & Alert Evaluation
Alertmanager	9093	http://localhost:9093	Predictive Alert Notifications
Grafana	3000	http://localhost:3000	Visual Infrastructure Dashboards
Locust UI	8089	http://localhost:8089	Synthetic Load Testing Controller